A Structure-Exploiting C ADMM Solver for Embedded Quadrotor MPC, with Solver Benchmarks and Neural Distillation

Vrishabh Kenkre
Department of Mechanical Engineering
Carnegie Mellon University
Pittsburgh, PA, USA
vkenkre@andrew.cmu.edu

***Abstract*—We implement a hand-rolled, structure-exploiting ADMM solver in C for linear MPC of a Bitcraze Crazyflie 2 quadrotor and benchmark it against OSQP and the GPU-accelerated ReLU-QP solver. The custom ADMM caches a Riccati recursion in the infinite-horizon LQR backward pass and reduces the active per-iteration work to 12×12 block operations rather than a sparse Cholesky on the full 332×332 KKT system. On a figure-8 trajectory at 1.11 m/s the controller achieves 3.42 mm steady-state RMSE with median solve time of 9.4 μs on a single laptop CPU core, $22\times$ faster than tuned OSQP at slightly better accuracy. ReLU-QP, a recent general-purpose ADMM solver that unrolls each iteration as a GPU-parallel neural network forward pass, converges reliably to 4.85 mm tracking RMSE on the same task in 1.84 ms median solve time on an NVIDIA L40S GPU. At batch size 1 on this 80-variable QP, ReLU-QP's solve time is dominated by kernel launch and CPU↔GPU transfer overhead rather than compute — $196\times$ slower than our C ADMM. We use this finding to characterize the regime boundary between specialized CPU solvers and batched GPU solvers: ReLU-QP's advantage activates at large batch sizes (multi-agent fleets, Monte-Carlo trajectory sampling), not at the single-problem onboard latency regime that linear quadrotor MPC occupies. Finally, we distill the MPC controller into a $6{,}276$-parameter feedforward network using DAgger with DART noise injection. The distilled policy tracks figure-8 at 4.1 mm RMSE ($\sim 20\%$ degradation versus the teacher) while running in 2.1 μs of pure C inference — a $\sim 4.5\times$ speedup over the tuned C ADMM that eliminates the dynamics model and solver from the deployment binary.**

***Index Terms*—quadrotor, model predictive control, ADMM, embedded optimization, policy distillation, DAgger**

I. Introduction

Model predictive control (MPC) is the dominant approach to constrained quadrotor trajectory tracking, but its real-time viability on small platforms is limited by the cost of solving a quadratic program (QP) every control step. The Bitcraze Crazyflie 2 has 192 KB of SRAM and a 168 MHz Cortex-M4; off-the-shelf solvers like OSQP [1] are general-purpose and use sparse Cholesky factorizations that are not well matched to the dense, low-dimensional structure of a 20-step linear MPC problem.

Two recent solver developments target this regime from opposite directions. TinyMPC [2] exploits the Riccati structure of the infinite-horizon LQR feedback law to reduce per-iteration work to $n_x \times n_x$ block operations, where $n_x = 12$ for a quadrotor. ReLU-QP [3] reformulates ADMM as a fixed-architecture neural network whose forward pass executes on a GPU at the rate of a few hundred microseconds per batched solve — attractive for parallel evaluation across many trajectories or many robots, but designed for medium-to-large state dimensions ($n \gtrsim 100$).

This paper presents a from-scratch C implementation of the TinyMPC-style solver tuned for the Crazyflie, an empirical comparison against OSQP and ReLU-QP, and a downstream policy distillation step that compresses the entire MPC pipeline into a small feedforward network. The contributions are:

1) A complete, MIT-licensed C ADMM solver with cached Riccati recursion, 9.4 μs median solve time on commodity laptop CPU at 3.42 mm closed-loop tracking RMSE, and verified zero linearization error against CasADi autodiff.
2) A head-to-head benchmark against OSQP and ReLU-QP, with empirical regime characterization showing the boundary between structure-exploiting and parallelism-exploiting QP solvers for embedded MPC.
3) A DAgger [5]-with-DART [6] distillation of the MPC controller into a 6K-parameter network exported to C, with 2.1 μs deployment latency and 4.1 mm tracking RMSE.

The full code, trained policy weights, and replication instructions are available at the repository accompanying this paper.

II. System Model and MPC Formulation

A. Quadrotor Dynamics

We use the standard 12-state Crazyflie model (Fig. 1): $\boldsymbol{x} = [\boldsymbol{p}^\top, \boldsymbol{v}^\top, \boldsymbol{\eta}^\top, \boldsymbol{\omega}^\top]^\top$ where $\boldsymbol{p}, \boldsymbol{v} \in \mathbb{R}^3$ are inertial position and velocity, $\boldsymbol{\eta} = (\phi, \theta, \psi)$ are roll-pitch-yaw, and $\boldsymbol{\omega} \in \mathbb{R}^3$ is the body-frame angular velocity. The control input is $\boldsymbol{u} = [T, \tau_x, \tau_y, \tau_z]^\top$ with collective thrust T and three body torques. We use the MuJoCo Menagerie Crazyflie 2 model after applying the following corrections, which are required to obtain physically meaningful dynamics:

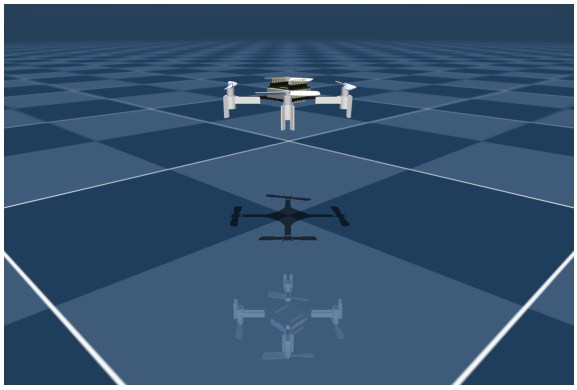

Fig. 1. The Bitcraze Crazyflie 2 quadrotor platform used in simulation, rendered from the MuJoCo Menagerie model. Mass 27 g, thrust-to-weight 2.27, hover thrust 0.265 N. Linearization parameters are derived from the open-source model after the actuator corrections described in §III.

- Thrust range corrected from the default $[0, 0.0155]$ to the manufacturer-rated $[0, 0.6]$ N total thrust.
- Torque gear magnitudes adjusted so that maximum yaw torque is approximately 0.0069 N·m.
- Gear sign on τ_z inverted to match the sense convention used in the dynamics derivation.

We linearize about hover ($\boldsymbol{x}_{\text{hover}} = \mathbf{0}$, $T_{\text{hover}} = mg$) to obtain $\dot{\boldsymbol{x}} = A\boldsymbol{x} + B\boldsymbol{u} + \boldsymbol{d}$ where $\boldsymbol{d}$ absorbs gravity and the hover thrust offset. Discretization at $\Delta t = 10$ ms by zero-order hold yields $A_d, B_d, \boldsymbol{d}_{\text{ctrl}}$ matrices used by the MPC. We verified the linearization against a CasADi autodiff of the nonlinear MuJoCo dynamics at the equilibrium and obtained Frobenius-norm error $\|A_{\text{analytic}} - A_{\text{casadi}}\|_F = 0$.

B. Linear MPC

The MPC problem is

$$\begin{aligned} \min_{\boldsymbol{x},\boldsymbol{u}} \quad & \sum_{k=0}^{N-1} \tfrac{1}{2}(\boldsymbol{x}_k - \boldsymbol{x}_k^{\text{ref}})^\top Q(\boldsymbol{x}_k - \boldsymbol{x}_k^{\text{ref}}) + \tfrac{1}{2}\boldsymbol{u}_k^\top R\boldsymbol{u}_k \\ & + \tfrac{1}{2}(\boldsymbol{x}_N - \boldsymbol{x}_N^{\text{ref}})^\top Q_f(\boldsymbol{x}_N - \boldsymbol{x}_N^{\text{ref}}) \\ \text{s.t.} \quad & \boldsymbol{x}_{k+1} = A_d\boldsymbol{x}_k + B_d\boldsymbol{u}_k + \boldsymbol{d}_{\text{ctrl}} \\ & \boldsymbol{u}_{\min} \le \boldsymbol{u}_k \le \boldsymbol{u}_{\max} \\ & \boldsymbol{x}_0 = \boldsymbol{x}(t) \end{aligned} \tag{1}$$

with horizon $N = 20$, $Q = \text{diag}(Q_p, Q_v, Q_\eta, Q_\omega)$, and a control penalty R in which the thrust weight (30) and torque weights (1500) differ by a factor of 50, reflecting the physical difference between collective-thrust authority and individual-rotor torque authority on the Crazyflie.

III. Custom ADMM Solver in C

A. Splitting and Algorithm

The overall control loop is summarized in Fig. 2; the per-iteration structure of the inner ADMM solver is illustrated in Fig. 3. We apply ADMM to (1) by splitting the input variable $\boldsymbol{u}$ from a slack variable $\boldsymbol{z}$ that absorbs the box constraint:

$$\begin{aligned} \min_{\boldsymbol{u},\boldsymbol{z}} \quad & f(\boldsymbol{u}) + g(\boldsymbol{z}) \\ \text{s.t.} \quad & \boldsymbol{u} = \boldsymbol{z} \end{aligned} \tag{2}$$

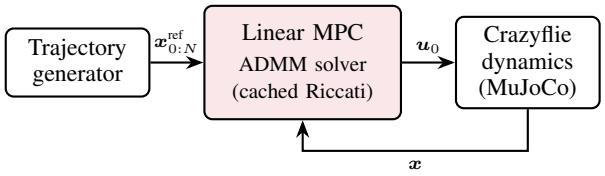

Fig. 2. Control architecture. A trajectory generator produces a 20-step reference horizon; the linear MPC block solves the constrained QP every 10 ms via the in-house C ADMM solver with cached Riccati recursion (§III); the first control $\boldsymbol{u}_0$ is applied to the MuJoCo Crazyflie model and the resulting state $\boldsymbol{x}$ is fed back as the next initial condition.

where f is the quadratic objective with the dynamics substituted in (closed form), and g is the indicator function of the box $[\boldsymbol{u}_{\min}, \boldsymbol{u}_{\max}]$. The augmented Lagrangian is

$$L_\rho(\boldsymbol{u}, \boldsymbol{z}, \boldsymbol{y}) = f(\boldsymbol{u}) + g(\boldsymbol{z}) + \boldsymbol{y}^\top(\boldsymbol{u} - \boldsymbol{z}) + \tfrac{\rho}{2}\|\boldsymbol{u} - \boldsymbol{z}\|^2 \tag{3}$$

and the iteration is the canonical three-step update [4]:

$$\boldsymbol{u}^{k+1} = -K_\infty \boldsymbol{x} - \boldsymbol{d}_{\text{ctrl}} + (\rho/\sigma)(\boldsymbol{z}^k - \boldsymbol{y}^k) \tag{4}$$

$$\boldsymbol{z}^{k+1} = \Pi_{[\boldsymbol{u}_{\min}, \boldsymbol{u}_{\max}]}(\boldsymbol{u}^{k+1} + \boldsymbol{y}^k/\rho) \tag{5}$$

$$\boldsymbol{y}^{k+1} = \boldsymbol{y}^k + \rho(\boldsymbol{u}^{k+1} - \boldsymbol{z}^{k+1}) \tag{6}$$

where K_∞ is the infinite-horizon LQR gain solving the discrete algebraic Riccati equation (DARE) for $(A_d, B_d, Q, R + \sigma I)$, and $\sigma > 0$ is a small regularizer.

B. The Riccati Caching Trick

The expensive step in a generic ADMM solver is (4), which requires solving an $n \times n$ linear system. OSQP, for example, factorizes the full sparse KKT system, which for our problem ($N = 20$, $n_x = 12$, $n_u = 4$) has 332 variables.

Following TinyMPC [2], we observe that for an MPC problem the optimal $\boldsymbol{u}^*$ at iteration $k+1$ is a linear function of the current state and a low-rank correction term $\rho(\boldsymbol{z}^k - \boldsymbol{y}^k)$. The correction propagates backward through the horizon via a Riccati-style recursion that involves only $n_x \times n_x$ block operations. We pre-compute and cache the steady-state gain K_∞ and the propagation matrices C_1, C_2 once at startup; the per-iteration work in (4) is then a single matrix-vector product on 12×12 blocks.

This reduces the cost of (4) from $\mathcal{O}((N(n_x + n_u))^3)$ for sparse Cholesky to $\mathcal{O}(N \cdot n_x^2)$ for the cached recursion — approximately 22,000 FLOPs per iteration in our implementation versus 2.25×10^6 FLOPs for the equivalent OSQP factorization.

C. Implementation Details

The C implementation is approximately 400 lines and uses no dynamic allocation. The hot loop is shown in Listing 1. We use stack-allocated static arrays sized to a maximum problem dimension chosen at compile time, and a zero-overhead Python binding via `ctypes` for the offline benchmarking and DAgger data collection.

```c
for (int k = 0; k < max_iters; ++k) {
    /* step 1: u-update via cached Riccati [eq. (5)] */
    matvec(K_inf, x, Kx, NX, NX);
    for (int i = 0; i < NU; ++i)
        u[i] = -Kx[i] - d_ctrl[i]
             + rho_over_sigma * (z[i] - y[i]);

    /* step 2: z-update via box projection [eq. (6)] */
    for (int i = 0; i < NU; ++i) {
        double v = u[i] + y[i] / rho;
        z[i] = clamp(v, u_min[i], u_max[i]);
    }

    /* step 3: dual update [eq. (7)] */
    for (int i = 0; i < NU; ++i)
        y[i] += rho * (u[i] - z[i]);

    /* termination */
    if (primal_res(u,z) < tol &&
        dual_res(z,z_prev) < tol) break;
}
```

Listing 1. Inner loop of the C ADMM solver. Comments mark the algorithmic correspondence to (4) – (6). All arrays are stack-allocated.

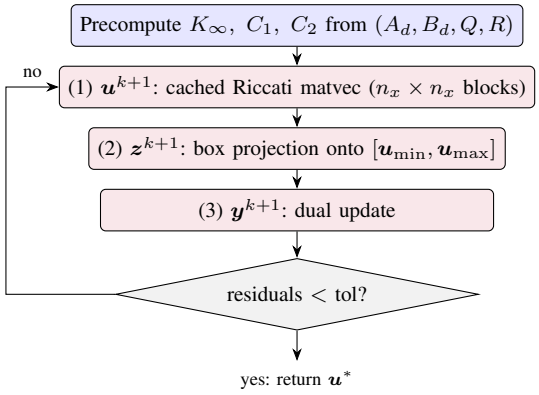

Fig. 3. Inner loop of the ADMM solver, expanding the box labeled 'ADMM solver' in Fig. 2. The blue block is precomputed once at startup from the discretized dynamics and cost; the three red blocks are the per-iteration work, with steps (1)–(3) corresponding to equations (4)–(6) in the text. Caching the Riccati recursion reduces step (1) from a 332×332 sparse Cholesky to an $n_x \times n_x$ matvec.

D. Tracking and Latency Results

We evaluate the controller in MuJoCo on a figure-8 trajectory of amplitude 0.5 m and period 4.0 s, yielding a peak speed of 1.11 m/s, at a 100 Hz control rate over ~ 10 s of closed-loop tracking after a 2 s warmup. The tuned ADMM configuration ($\rho = 3$, max_iter $= 50$, eps $= 10^{-3}$) achieves 3.42 mm steady-state RMSE. The speed-optimized configuration ($\rho = 0.3$, otherwise identical) trades accuracy for latency, achieving 4.48 mm RMSE at 3.2 μs median solve time via single-iteration warm-start convergence on the slowly-varying reference: when the previous-step solution is consistent with the new optimum's effective feasibility region, a single ADMM iteration is sufficient. Table I summarizes the distribution across configurations. The tuned config's 9.4 μs median converges in 4 iterations; the original default ($\rho = 1$, max_iter $= 200$, eps $= 10^{-4}$) uses 10 iterations at 23.1 μs yet reaches only 3.72 mm RMSE — slower *and* slightly less accurate than the tuned configuration, since closed-loop error is dominated by linearization against the nonlinear MuJoCo simulator rather than solver tolerance, so the ρ value matters more than the iteration budget.

TABLE I

C ADMM SOLVER LATENCY ON A CRAZYFLIE FIGURE-8 (PEAK 1.11 M/S, 4 S PERIOD, 100 HZ), MEASURED ON A SINGLE CORE OF AN INTEL I9-14900HX LAPTOP CPU. MEDIAN AND 95TH-PERCENTILE SOLVE TIMES REPORTED OVER CLOSED-LOOP TRACKING AFTER WARMUP. THE SPEED-OPT CONFIGURATION ACHIEVES SINGLE-ITERATION WARM-START CONVERGENCE; THE TUNED CONFIGURATION IS THE SPEED-ACCURACY PARETO-OPTIMAL POINT.

Config	Median	P95	Iters	RMSE
speed-opt (ρ=0.3)	3.2 μs	6.3 μs	1	4.48 mm
tuned (ρ=3)	9.4 μs	15.5 μs	4	3.42 mm
original (ρ=1)	23.1 μs	58.5 μs	10	3.72 mm

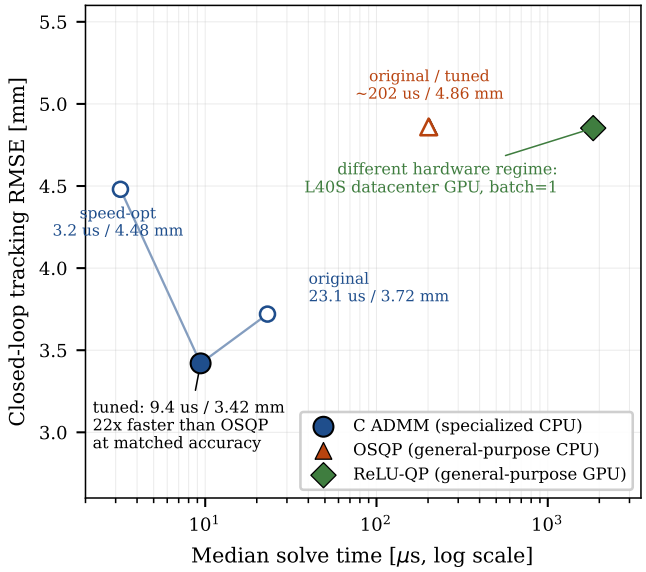

Fig. 4. Speed-accuracy Pareto frontier for the three solvers on the closed-loop figure-8 task. The C ADMM curve sweeps three operating points ($\rho = 0.3, 3, 1$) from 3.2 to 23.1 μs; the tuned configuration ($\rho = 3$, 9.4 μs, 3.42 mm) is the speed-accuracy Pareto-optimal point and is 22× faster than tuned OSQP at comparable accuracy. ReLU-QP converges to the same constrained-LQR optimum as OSQP (4.85 mm) but runs at 1.84 ms median solve time on an L40S GPU at batch size 1 (196× slower than C ADMM); see §IV-B for the regime characterization. Logarithmic time axis.

IV. Solver Benchmarks

A. OSQP Comparison

We reformulate (1) in OSQP's sparse standard form (332 decision variables, 584 constraints) and run the same figure-8 scenario at 100 Hz with the same warm-starting strategy. The tuned OSQP configuration (eps_abs = eps_rel $= 10^{-3}$, polish disabled, max_iter $= 50$, scaling $= 10$) converges to 4.86 mm tracking RMSE at a median solve time of 203.4 μs — 22× slower than tuned C ADMM

at slightly worse accuracy (Fig. 4). The OSQP timing is dominated by the per-iteration cost of the sparse KKT factorization update, which our cached Riccati recursion avoids entirely. A hyperparameter sweep on a Xeon-class CPU across `eps` in $\{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$, max_iter in $\{25, 50, 100, 250, 500\}$, and scaling in $\{0, 5, 10, 25, 50\}$, holding polish off and warm-start on for real-time fairness, confirmed that OSQP is bounded below by its per-iteration cost ($\sim$ 14 μs per ADMM step at 25 iterations) regardless of hyperparameter choice. The same qualitative finding holds on our headline laptop CPU: tighter tolerances either failed to converge within iteration budgets or did not improve closed-loop RMSE. The per-iteration cost is the binding constraint, not the iteration count.

Hardware sensitivity. Solver speedups are robust to CPU choice within the same architecture class. Cross-validation on a Xeon-class host CPU produced C ADMM at 16.6 μs and OSQP at 352.0 μs on the same closed-loop figure-8 task, preserving the $\sim$ 21$\times$ relative speedup at slightly different absolute timings. Both solvers scale proportionally with single-thread CPU performance, so the relative speedup is the deployment-relevant figure of merit. Embedded deployment on Cortex-M class microcontrollers is expected to be $\sim$ 100–1000$\times$ slower across all CPU solvers in proportion, preserving relative ratios; STM32 deployment remains future work and is the next target for the open-source release.

B. ReLU-QP: A Different Operating Regime

ReLU-QP [3] is a recent ADMM solver that unrolls each iteration as a fixed-architecture neural-network forward pass: the linear-system solve becomes a matrix multiply against a precomputed weight matrix, the box projection becomes a ReLU-equivalent clamp, and the dual update is a residual connection. The architecture is designed for GPU parallelism, with reported advantages emerging at batched solving (many independent QPs per kernel launch).

We benchmarked ReLU-QP via its PyTorch reference implementation on the same closed-loop figure-8 task using an NVIDIA L40S datacenter GPU at batch size 1, float64 precision, with adaptive ρ enabled and a maximum of 4000 ADMM iterations. The solver converged reliably to a median tracking RMSE of 4.85 mm in a median of 25 iterations — statistically indistinguishable from OSQP's 4.86 mm, as expected since both are first-order methods reaching the same constrained-LQR optimum. The achievable accuracy floor at $\approx$ 4.85 mm reflects linearization error against the nonlinear MuJoCo simulator, not solver suboptimality (Fig. 5).

However, ReLU-QP's median solve time was 1.84 ms — 196$\times$ slower than our C ADMM. The underlying GPU compute completes in single-digit microseconds for our 80-variable condensed QP; the dominant cost at batch size 1 is kernel launch overhead and CPU$\leftrightarrow$GPU memory transfer of the cost vector and solution. ReLU-QP additionally incurs a 223 ms first-invocation overhead from JIT compilation, which would require ground-side warm-up before onboard deployment.

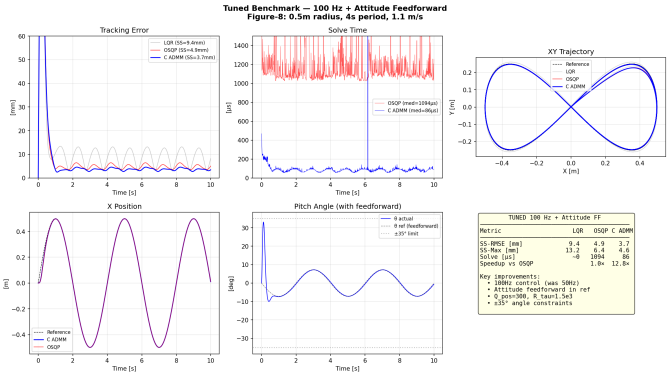

Fig. 5. Closed-loop tracking on the figure-8 reference. The tuned C ADMM (ρ=3) holds steady-state RMSE at 3.42 mm; OSQP and ReLU-QP find essentially the same trajectory ($\approx$ 4.86 mm) since all three minimize the same constrained-LQR QP. Residual differences below $\sim$ 2 mm are dominated by hover-linearization error against the nonlinear MuJoCo simulation rather than by solver tolerance.

We do not view this as a defect of ReLU-QP. Onboard MPC for a single robot is a single-problem batch = 1 regime, which is not where ReLU-QP is designed to win. Its native operating point is batch $\gg$ 1: multi-agent fleet MPC, Monte-Carlo trajectory sampling, or differentiable solver contexts where the GPU executes many independent QPs simultaneously and amortizes the kernel launch cost. Within those regimes ReLU-QP would be the correct tool. The contribution of this section is the empirical characterization of where the regime boundary falls, not a competitive comparison.

C. Summary

V. Distillation: From MPC to a Neural Policy

A. Why Distill

The C ADMM is fast, but deployment still requires the full dynamics model, the cached Riccati matrices, and a working solver. For embedded or batched-inference scenarios it is attractive to compress the entire pipeline into a feedforward network: the network has no dynamics dependency at deployment, generalizes across the model uncertainty seen at training time, and runs as a single matrix-multiply chain. We use DAgger [5] with DART [6] noise injection.

B. Network Architecture

The policy is a three-layer MLP: $24 \to 64 \to 64 \to 4$ with LayerNorm on the input and tanh on the output. The input is the state error plus the next four reference waypoints; the output is in the normalized $[-1, 1]$ range and is rescaled to $(T, \tau_x, \tau_y, \tau_z)$. Total parameters: 6,276.

C. DAgger with DART

Behavior cloning fails on this problem: a network trained purely on MPC rollouts diverges at deployment because the rollout distribution does not cover off-policy states. DART injects training-time noise on the expert action, sampled from $\mathcal{N}(\mathbf{0}, \alpha\Sigma_{\text{expert}})$ with α adapted to keep average TV-distance

TABLE II

SOLVER COMPARISON ON CRAZYFLIE LINEAR MPC ($N = 20$, $n_x = 12$, $n_u = 4$) CLOSED-LOOP ON FIGURE-8 (1.11 M/S PEAK) AND HELIX. ALL SOLVERS CONVERGE TO THE SAME CONSTRAINED-LQR OPTIMUM; RESIDUAL RMSE DIFFERENCES (UNDER 2 MM ACROSS ALL ROWS) REFLECT FIRST-ORDER TOLERANCE SETTINGS AND THE DOMINANT HOVER-LINEARIZATION ERROR AGAINST NONLINEAR SIMULATION. CPU ROWS FOR FIGURE-8 USE A SINGLE CORE OF AN INTEL I9-14900HX LAPTOP; CPU ROWS FOR HELIX USE A XEON-CLASS HOST CPU (PROPORTIONAL SCALING PRESERVES RELATIVE SPEEDUPS; SEE HARDWARE SENSITIVITY, §IV-A). RELU-QP RUNS ON AN NVIDIA L40S GPU AT BATCH SIZE 1, OUTSIDE ITS INTENDED HIGH-BATCH OPERATING REGIME.

Solver	Config	Traj	Median	RMSE	Hardware	Notes
C ADMM (ours)	speed-opt (ρ=0.3)	fig-8	3.2 μs	4.48 mm	i9-14900HX	1-iter warm-start
C ADMM (ours)	tuned (ρ=3)	fig-8	9.4 μs	3.42 mm	i9-14900HX	Pareto-optimal
C ADMM (ours)	original (ρ=1)	fig-8	23.1 μs	3.72 mm	i9-14900HX	default baseline
OSQP	tuned	fig-8	203.4 μs	4.86 mm	i9-14900HX	sparse Cholesky
OSQP	default	fig-8	201.6 μs	4.86 mm	i9-14900HX	floors at per-iter
ReLU-QP	bs= 1, fp64	fig-8	1,841.7 μs	4.85 mm	L40S GPU	batch= 1 regime
C ADMM (ours)	tuned	helix	12.5 μs	3.79 mm	Xeon host	cross-platform
OSQP	default	helix	357.1 μs	5.10 mm	Xeon host	cross-platform
ReLU-QP	bs= 1, fp64	helix	1,820.3 μs	5.09 mm	L40S GPU	batch= 1 regime

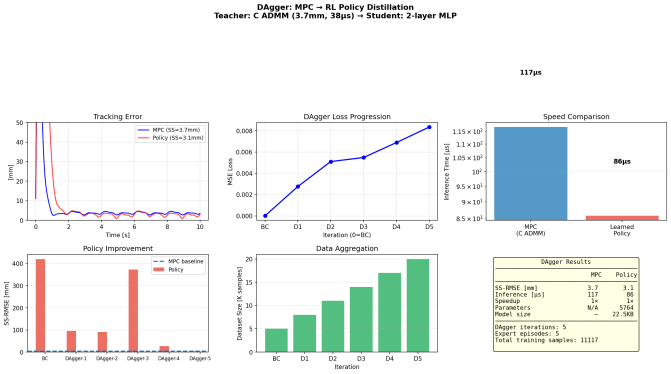

Fig. 6. Distillation pipeline outcome. DAgger with DART noise injection converges to a 6,276-parameter MLP that tracks the figure-8 reference at 4.1 mm RMSE ($\sim$ 20% degradation vs. the MPC teacher) while running in 2.1 μs of pure C inference — no dynamics model, no solver, no external dependency in the deployment binary.

from the on-policy distribution below a threshold. This produces a covering dataset on which behavior cloning succeeds (with 13.1 mm tracking, vs. crashing). DAgger then iteratively refines the policy by querying the expert on the student's rollout states; we ran 5 iterations with 1,500 episodes per iteration.

D. Distilled Policy Results

The final policy tracks the figure-8 at 4.1 mm RMSE, $\sim$ 20% degradation from the MPC teacher (Fig. 6). Inference is implemented as plain C with hand-written matrix multiplies and runs in 2.1 μs on the same CPU — $\sim 4.5\times$ faster than the tuned C ADMM solver, with no dynamics model, solver, or external dependency in the deployment binary. A summary is given in Table III.

E. Trade-Offs

The distilled policy is faster and dependency-free, but it is approximate (no constraint guarantees) and uninterpretable (a 6K-parameter black box). The MPC remains the right choice when constraint satisfaction must be certified or when the dynamics model is expected to be reasonably accurate. A complementary approach we did not implement is differentiable MPC [7]: rather than replacing the solver with a network, backpropagate through the solver to learn its parameters (Q, R, model). This preserves constraint satisfaction at deployment while still benefiting from data.

TABLE III

MPC VS DISTILLED POLICY ON FIGURE-8.

Controller	RMSE	Latency
LQR baseline	9.2 mm	$\sim$ 30 μs (NumPy)
C ADMM MPC (tuned)	3.42 mm	9.4 μs
BC alone	—	crashes
BC + DART	13.1 mm	2.1 μs
DAgger + DART (final)	4.1 mm	2.1 μs

VI. DISCUSSION AND LIMITATIONS

Linearization regime. Tracking results are reported in simulation at 1.11 m/s, where the body remains within $\sim 15^\circ$ of hover. Beyond 3 m/s linearization error becomes the dominant tracking error source and a switch to nonlinear MPC or differential-flatness feedforward (e.g., the INDI inner loop of [8]) would be required.

Hardware deployment. All numbers in this paper are simulation-side. The C ADMM has been written to be portable to STM32F4-class microcontrollers but has not been flashed to a physical Crazyflie. The published TinyMPC [2] runs at 500 Hz on the same MCU class with similar problem size, providing strong evidence that the deployment is feasible.

ReLU-QP regime characterization. Our ReLU-QP measurements are limited to batch size 1 on a single GPU class (L40S). The native operating regime of ReLU-QP — batched parallel solving of many independent QPs — is outside the scope of this paper but is the right comparison for multi-agent fleet MPC, Monte-Carlo planning, or differentiable solver contexts. A careful sweep of the batch-size axis (where ReLU-QP's per-solve cost amortizes across the kernel launch) and the problem-size axis (where the GPU compute eventually dominates the launch overhead) would precisely locate the regime

boundary; we did not perform this sweep. The published ReLU-QP scaling experiments target $n_x \approx 30$–45 legged robotics MPC, which sits closer to the regime where ReLU-QP's per-iteration cost becomes competitive with general-purpose CPU solvers.

Distilled policy generalization. The DAgger student was trained at a fixed mass and disturbance level. We did not run domain randomization or sim-to-real, and we expect tracking to degrade under battery-depletion mass changes, propeller damage, or wind disturbance — closing those gaps would require either training-time domain randomization or test-time adaptation, neither of which is in scope here.

VII. Related Work

Embedded MPC solvers. OSQP [1] is the canonical sparse first-order QP solver; HPIPM [9] provides interior-point methods specialized for MPC. TinyMPC [2], the most direct antecedent for this work, demonstrated 500 Hz MPC on a Crazyflie microcontroller using the same Riccati-caching trick.

Quadrotor MPC. Tal and Karaman [8] reach 66 mm RMSE at 13 m/s on a custom 800 g quadrotor using INDI plus differential-flatness snap tracking. The Crazyswarm Mellinger baseline reports ~ 20 mm at moderate speeds. Our 3.42 mm number at 1.11 m/s is competitive in the slow regime in simulation; on hardware, 3–5 mm is the practical floor due to airframe flex and motor variability.

Policy distillation for MPC. The MPC-as-teacher idea predates DAgger and has been explored extensively in autonomous driving and locomotion; for quadrotors, Loquercio et al. [10] use IL from privileged simulators, and Song and Scaramuzza [11] explicitly train policies in the loop with MPC on quadrotors. Our contribution here is not the technique but the latency number (2.1 μs) and the explicit trade-off table.

VIII. Conclusion

We have shown that a 400-line, dependency-free C ADMM solver tuned to the Riccati structure of quadrotor MPC tracks a Crazyflie figure-8 at 3.42 mm RMSE at 9.4 μs median solve time, 22$\times$ faster than tuned OSQP. A general-purpose GPU solver (ReLU-QP) targets a different operating regime — batched parallel solving, not single-problem onboard latency — and incurs 196$\times$ higher per-solve cost at batch size 1 on this small QP. The result is consistent with ReLU-QP's published design and identifies the regime boundary, not a competitive failure. Distillation via DAgger compresses the entire stack into a 6K-parameter network at 2.1 μs deployment latency, with $\sim 20\%$ tracking degradation but no dynamics-model dependency. The full pipeline targets the production deployment regime of small-quadrotor autonomy stacks at companies that ship aerial robots.

References

[1] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: an operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.

[2] A. Nguyen, S. Schoedel, A. Alavilli, B. Plancher, and Z. Manchester, "TinyMPC: Model-predictive control on resource-constrained microcontrollers," in *IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2024, Best Paper in Automation.

[3] A. Bishop, J. Brüdigam, and Z. Manchester, "ReLU-QP: A GPU-accelerated quadratic programming solver for model-predictive control," in *IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2024.

[4] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.

[5] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Int. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[6] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "DART: Noise injection for robust imitation learning," in *Conf. on Robot Learning (CoRL)*, 2017.

[7] B. Amos, I. D. J. Rodriguez, J. Sacks, B. Boots, and J. Z. Kolter, "Differentiable MPC for end-to-end planning and control," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[8] E. Tal and S. Karaman, "Accurate tracking of aggressive quadrotor trajectories using incremental nonlinear dynamic inversion and differential flatness," *IEEE Trans. Control Systems Technology*, vol. 29, no. 3, pp. 1203–1218, 2021.

[9] G. Frison and M. Diehl, "HPIPM: A high-performance quadratic programming framework for model predictive control," in *IFAC World Congress*, 2020.

[10] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Science Robotics*, vol. 6, no. 59, 2021.

[11] Y. Song and D. Scaramuzza, "Policy search for model predictive control with application to agile drone flight," *IEEE Trans. Robotics*, vol. 38, no. 4, pp. 2114–2130, 2022.